

# Introduction

CCP6224

Object-oriented Analysis and Design

# Course Learning Outcomes

|      |                                                                                          | Domain    | Level |
|------|------------------------------------------------------------------------------------------|-----------|-------|
| CLO1 | Explain object-oriented analysis and design concepts                                     | Cognitive | 2     |
| CLO2 | Apply proper object-oriented techniques to analyse and design software                   | Cognitive | 3     |
| CLO3 | Solve design problems by employing design patterns and object-oriented design techniques | Cognitive | 3     |

# Lecture outcomes

- At the end of this lecture you should be able to
  - Recall the concepts of object-oriented programming
  - Identify and use fundamental object-oriented terms and how they relate to coding
  - Differentiate between styles of programming languages
  - Understand the differences and similarities between OO programming languages

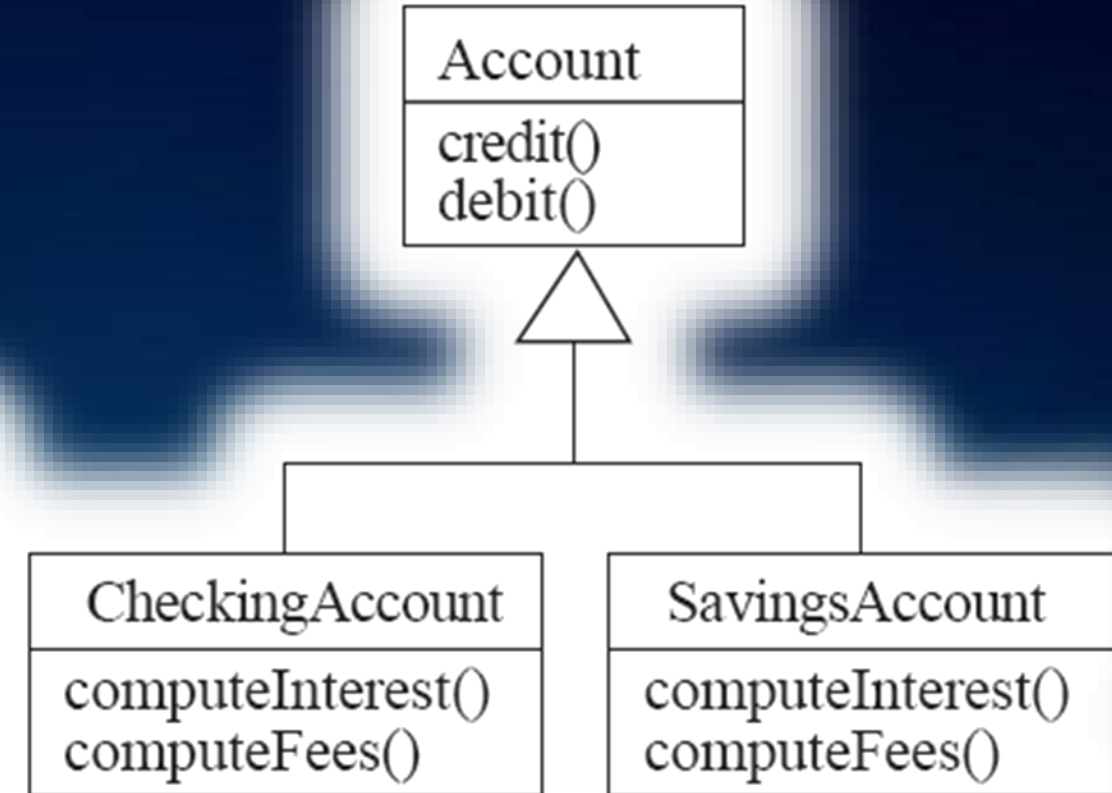
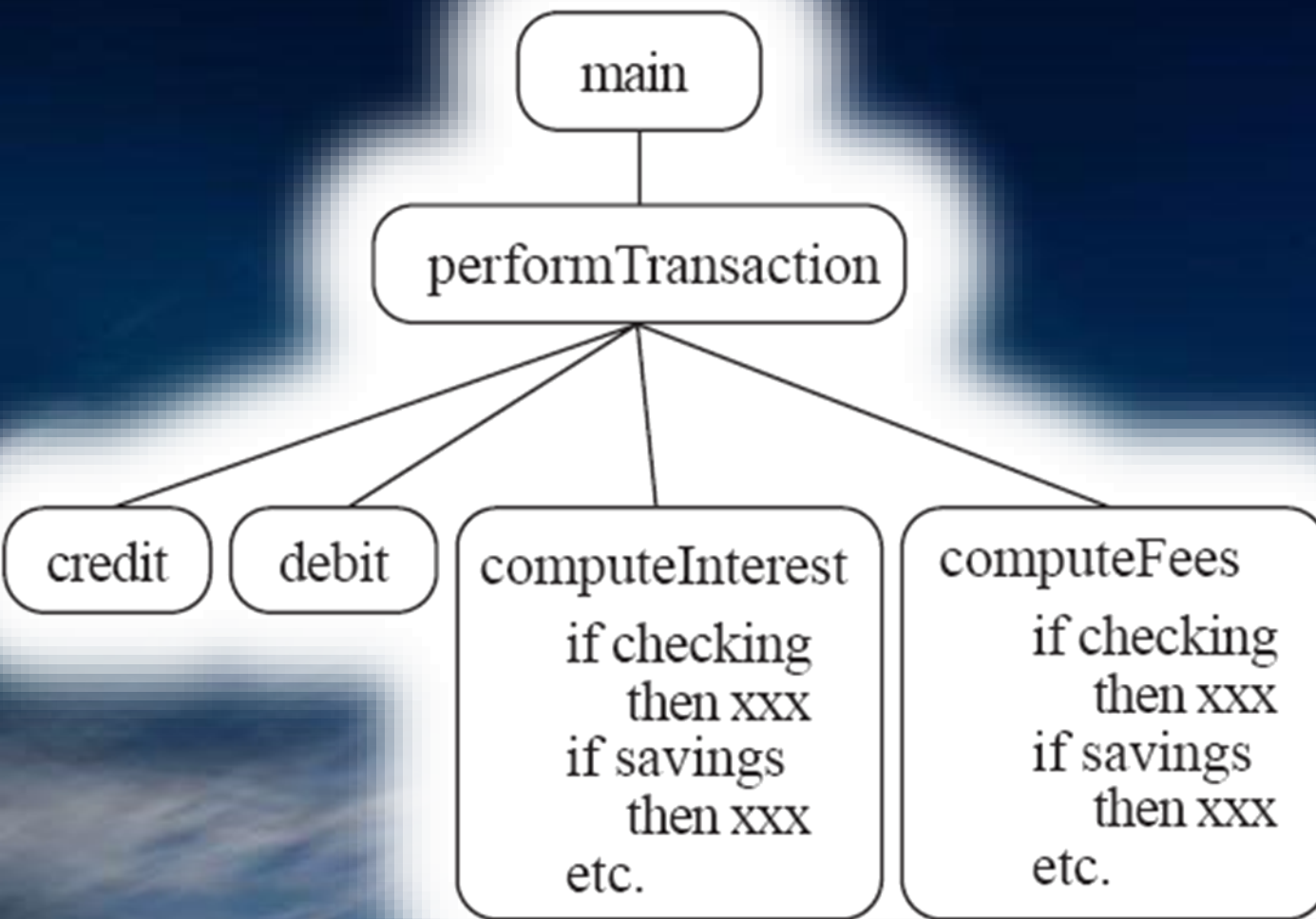
*What do you remember  
about OOPDS?*





# Object orientation

- Is a programming paradigm (style) found in modern programming languages
  - Procedural paradigm:
    - Software is organized around the notion of procedures/functions
    - Works as long as the data is simple
    - The program is typically modelled with a data flow diagram
  - Object oriented paradigm:
    - Revolves around objects and what can they do and how they can perform certain tasks.
    - The objects are instances of classes, which are data abstractions and contain procedural abstractions that operate on the objects themselves or other objects
    - A running OO program can be seen as a collection of objects collaborating to perform a given task



# What is an object?

- A chunk of structured data in a running software system
  - Has properties that represents its current state
  - Has behaviour which represents how it acts and reacts
- May simulate/mirror the behaviour of an object in the real world
- What are examples of objects?



Jane:

dateOfBirth="1955/02/02"  
address="99 UML St."  
position="Manager"

Savings account 12876:

balance=1976.32  
opened="1999/03/03"

Greg:

dateOfBirth="1970/01/01"  
address="75 Object Dr."

Margaret:

dateOfBirth="1984/03/03"  
address="150 C++ Rd."  
position="Teller"

Instant teller 876:

location="Java Valley Cafe"

Mortgage account 29865:

balance=198760.00  
opened="2003/08/12"  
property="75 Object Dr."

Transaction 487:

amount=200.00  
time="2001/09/01 14:30"



# What are classes?

- A unit of abstraction in an object oriented (OO) program
- Classes represents similar objects which are required in the program and its instances
- In program code, a class is a kind of software module
  - Describes its instances' structure (properties)
  - Contains methods to implement their behaviour
- What is the difference between *class* and *instance*? Examples?

# OO class naming convention

- The compiler won't care, but humans who read your program will:

- Use capital letters for classes

E.g. BankAccount not bankAccount

- Use singular nouns

E.g. Student not Students

- Use the right level of generality

E.g. Municipality, not City

- Try to make sure the name won't be misunderstood

E.g. 'amount' might not be clear which amount you are referring to.

# Object variables

- Variables defined *inside* a class corresponding to data present in each instance are called instance variables (also called fields or member variables)
  - Identifies the state of an object's instance
  - Used to represent attributes and/or associations
    - E.g. name, dateOfBirth, supervisor, coursesTaken
- A class variable is a value is shared by all instances of a class (Also called a *static variable*)
  - If one instance sets the value of a class variable, then all the other instances see the same changed value.
  - Class variables are useful for:
    - Default or 'constant' values (e.g. PI)
    - Shared incremental values
  - Caution: do not over-use class variables because it defeats OO approach of variable responsibility



# Object methods

- Method
  - A procedural abstraction used to implement the behaviour of a class – a ‘function’
- Several different classes can have methods with the same name
  - They implement the same abstract operation in ways suitable to each class
  - Using a common method name makes it easy to remember which function to call in different objects to achieve the same desired output
    - E.g. calculating area in a rectangle is done differently from a circle but each class may have the similarly named method ‘*calculateArea*’



# Polymorphism

- A property of object-oriented software by which an operation may be performed in different ways in different classes.
  - Requires that there be multiple methods of the same name
  - The choice of which one to execute depends on the object that is in a variable
  - Reduces the need for programmers to code many if-else or switch statements
- What were the conditions required to show methods with polymorphism using C++?



# Inheritance hierarchies

- Superclasses
  - Contain features common to a set of subclasses
  - Sometimes called parent class
- Subclasses
  - Inherits from superclasses but contains features unique to the subclass (specialized)
  - Sometimes called child class
- Inheritance hierarchies
  - Show the relationships among superclasses and subclasses
  - A triangle shows a generalization
- Inheritance
  - The implicit possession by all subclasses of features defined in its superclasses



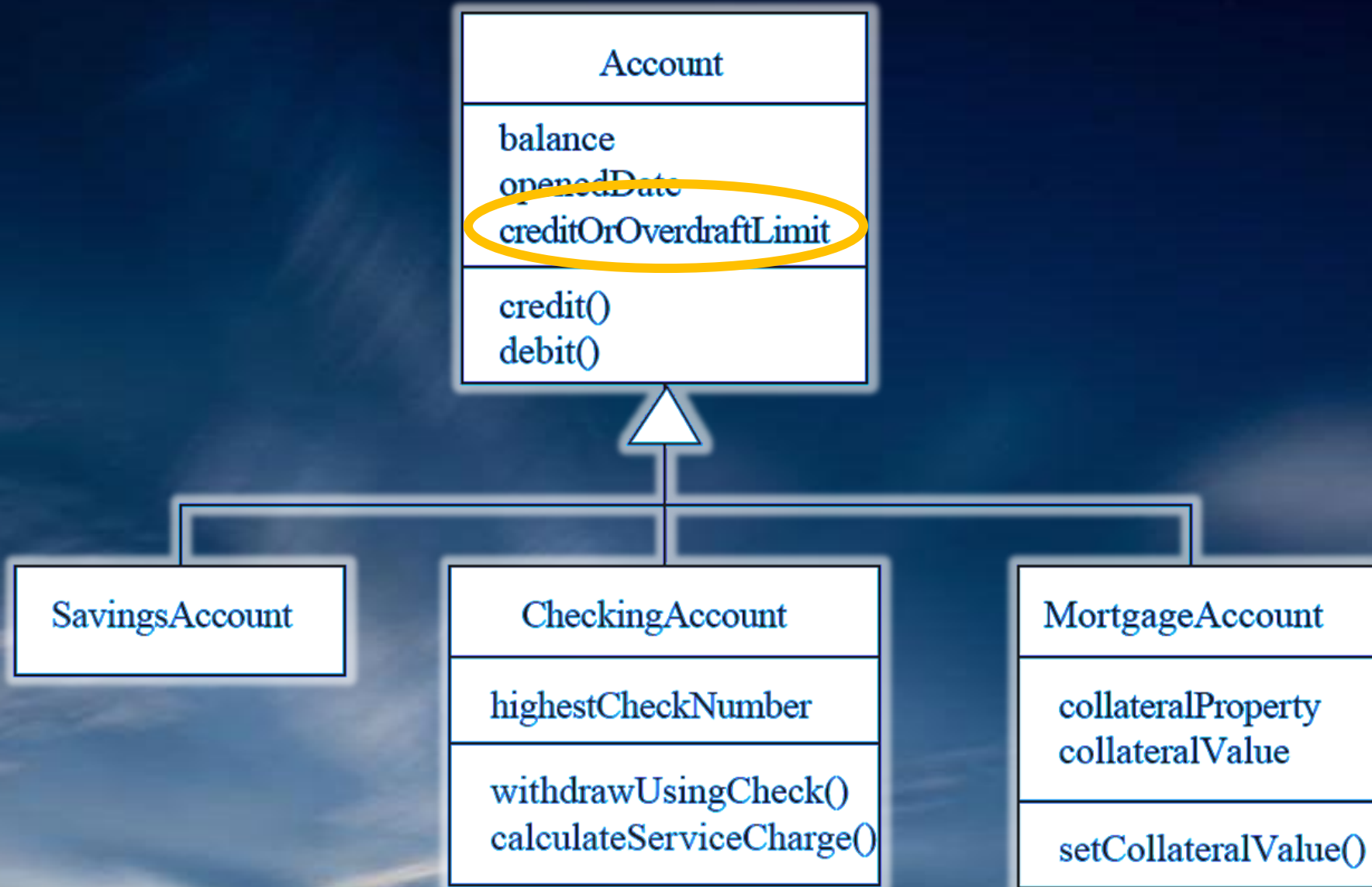
Always check generalizations to ensure they obey the is-a rule, e.g.:

*“A checking account **is a** (type of) account”*

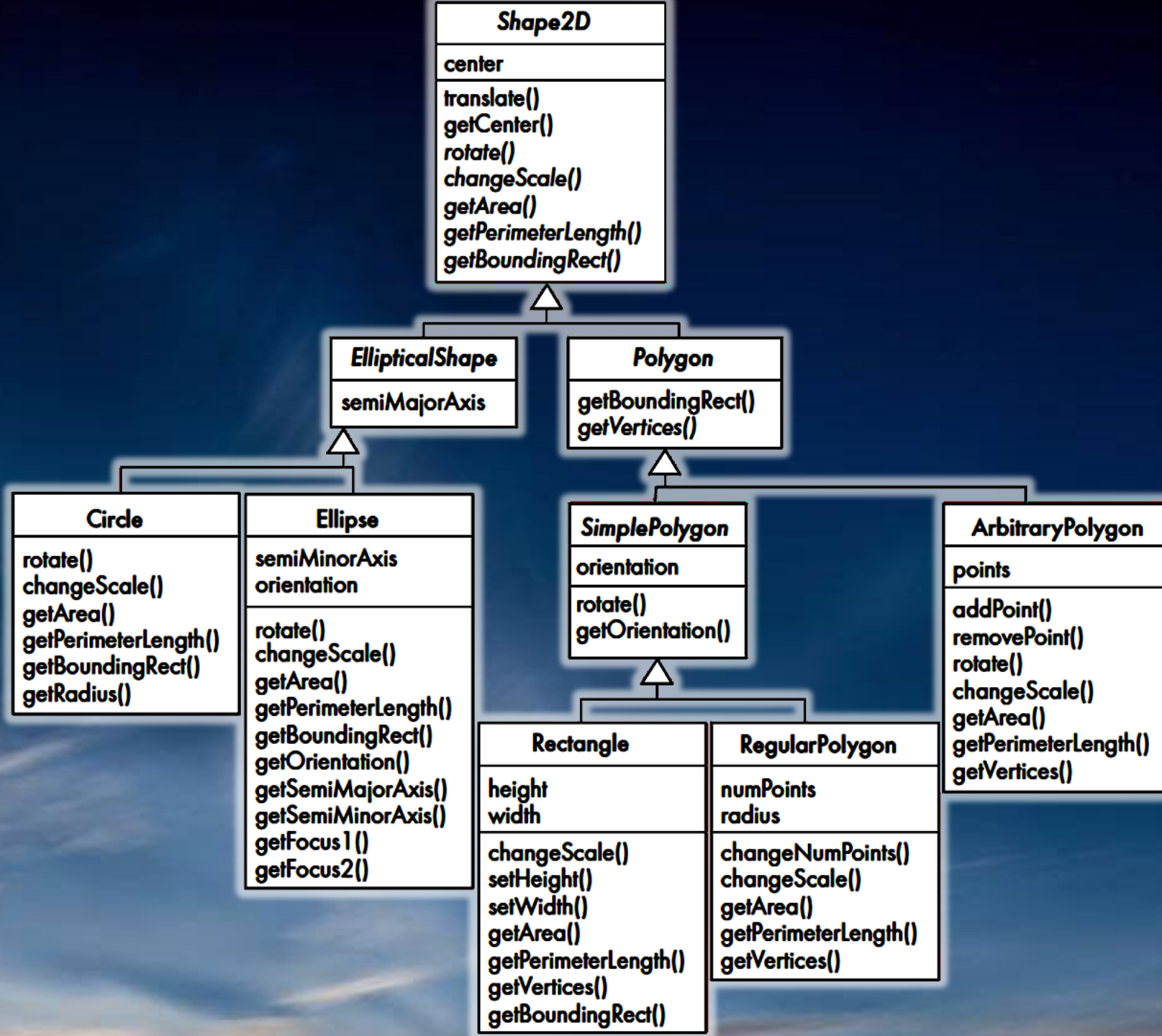
*“A student **is a** person”*



- Make sure inherited features make sense in subclasses







# Abstract classes

- Abstract classes (AC) are usually the highest in the inheritance hierarchy
- ACs have no implementation and their operations are abstract (method declarations without definitions)
- Because of this
  - No instances can be created
  - The opposite of an abstract class is a *concrete* class
- Classes that inherit from ACs MUST override ALL methods from the AC.
- If a superclass has an abstract operation, then its subclasses at some level must have a concrete method for the operation
  - Subclasses must have or inherit concrete methods for all operations
  - Leaf classes (final) must be concrete

# Overriding

- Applies to methods that are inherited from a parent class.
- A public method would normally be inherited, but with overriding the subclass contains a new version instead
  - For restriction
    - E.g. **scale(x,y)** would not work in **Circle** so the method is rewritten
  - For extension
    - E.g. **SavingsAccount** might charge an extra fee following every debit whilst other account types do not
  - For optimization
    - E.g. The **getPerimeterLength** method in **Circle** is much simpler than the one in **Ellipse**



The background of the image is a photograph of a clear blue sky with scattered, thin, white clouds. The clouds are more concentrated towards the bottom and sides, leaving a large clear area in the center where the text is placed.

*What about overloading??*



# Access control

- Applies to all content in a class
  - public
    - Any class can access
  - protected
    - Only code in the package, or subclasses can access
  - (blank)
    - Only code in the package can access
  - private
    - Only code written in the class can access
    - Inheritance still occurs!

# Concepts that define OO

The following are necessary for a system or language to be OO

- Identity
  - ✓ Each object is distinct from each other object, and can be referred to
  - ✓ Two objects are distinct even if they have the same data
- Classes
  - ✓ The code is organized using classes, each of which describes a set of objects
- Inheritance
  - ✓ The mechanism where features in a hierarchy inherit from superclasses to subclasses
- Polymorphism
  - ✓ The mechanism by which several methods can have the same name and implement the same abstract operation.

# Other key concepts

- Abstraction
  - Object → something in the world
  - Class → objects
  - Superclass → subclasses
  - Operation → methods
  - Attributes and associations → instance variables
- Modularity
  - Code can be constructed entirely of classes
- Encapsulation
  - Details can be hidden in classes
  - This gives rise to information hiding:
    - Programmers do not need to know all the details of a class



# Java backstory

- The first object oriented programming language was Simula-67
  - ✓ designed to allow programmers to write simulation programs
- In the early 1980's, Smalltalk was developed at Xerox PARC
  - ✓ New syntax, large open-source library of reusable code, bytecode, platform independence, garbage collection.
- Late 1980's, C++ was developed by B. Stroustrup,
  - ✓ Recognized the advantages of OO but also recognized that there were tremendous numbers of C programmers
- In 1991, engineers at Sun Microsystems started a project to design a language that could be used in consumer 'smart devices': Oak
  - ✓ When the Internet gained popularity, Sun saw an opportunity to exploit the technology.
  - ✓ The new language, renamed Java, was formally presented in 1995 at the SunWorld '95 conference.

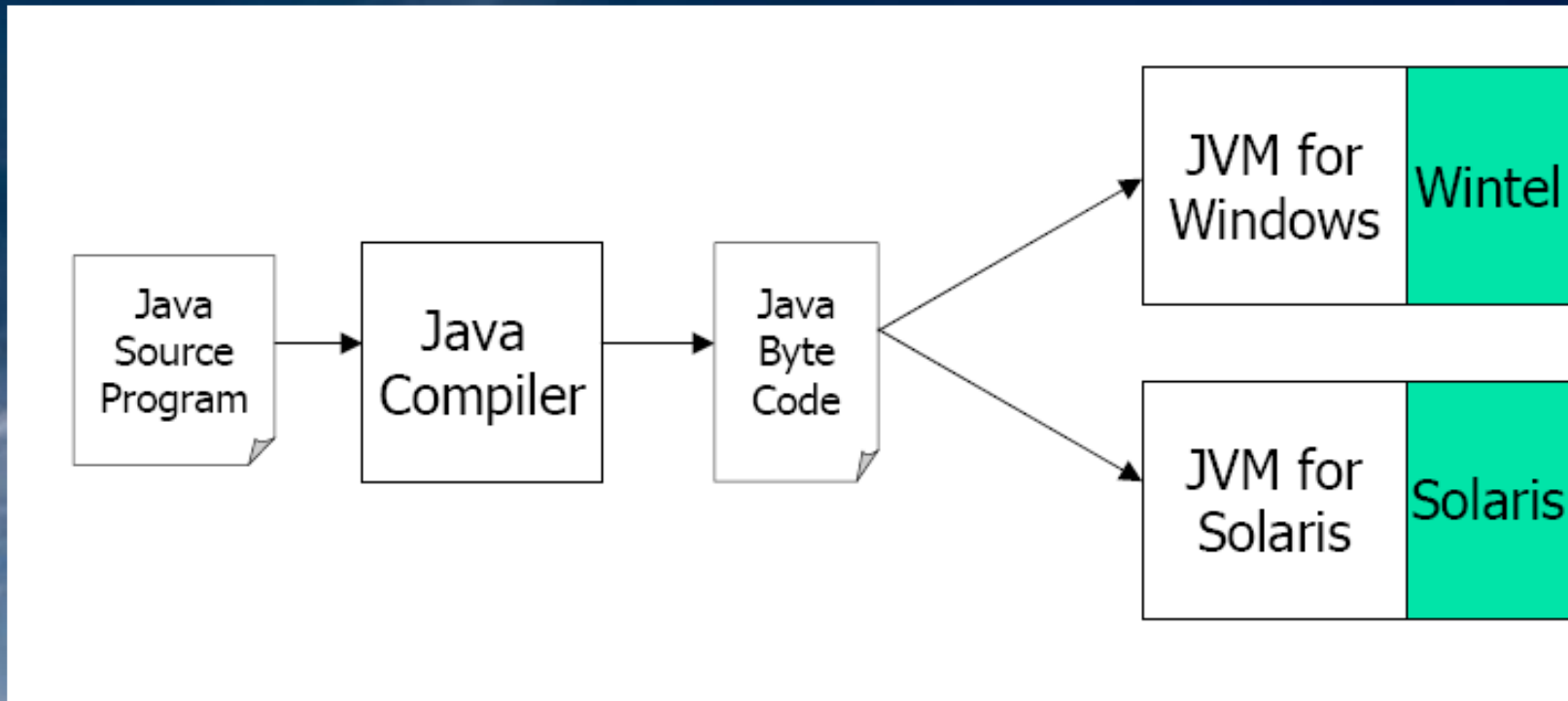


# Why Java?

- Java is an OO programming language which is:
  - simple (C,C++ syntax, single inheritance, auto-garbage-collection)
  - reliable (strong typing, no pointer-arithmetic, exception handling)
  - platform-independent (compiles to virtual machine)
  - Inherently multithreaded - you can create multithreaded programs (very useful for GUI or network apps)
  - Well documented libraries (called **packages** in Java)
  - Hardware support (Android mobile platform, media platform)
- Any OO language could be used for this course (e.g. C++, C#, Ruby, PHP, etc) but Java is chosen because majority of its syntax follows what you already have learnt from C++

# WORM

- Write Once, Run Many
- Java Programs compile to a Java Byte Code binary.
- All Java Virtual Machines (Windows JVM, Solaris JVM) understand Java Byte Code
- Therefore, Java is *relatively* platform independent



# Using Java

- To run Java, you need the Java Runtime Environment (JRE)
  - Specific for each platform
  - Executes compiled binaries in a virtualized environment
- To code in Java, you need the Java Development Kit (JDK)
  - Each JDK is specific for each platform.
  - Compiles Java code into byte code (binaries) for the target platform.



# Similarities between C++ and Java

- Most OO concepts are carried over from C++ - the syntax is just possibly different
- Basic programming concepts remain the same - control structures (loops, case, if-else etc), operators (relational, mathematical, set, logical, bitwise)
- Variables from C++ work similarly in Java (but their scope and capacity maybe different)
- Arrays work the same but has alternative declaration style
- Similar method of writing functions (return type, name + parameters)
- Naming styles are the same – CaSe SeNsItIvE too!

# Differences between C++ / Java

| C++                                           | Java                                          |
|-----------------------------------------------|-----------------------------------------------|
| Has preprocessor statements (include, define) | No preprocessor (no #includes ....)           |
| Has enum                                      | No enums but has named constraints            |
| Can have standalone functions                 | All functions MUST belong to a class          |
| Has global functions and data                 | No global functions or data                   |
| Can have multiple inheritance                 | No multiple inheritance                       |
| Inherits legacy goto commands                 | No goto but still supports break and continue |
| Operator overloading is allowed               | No operator overloading                       |
| No garbage collection                         | Automatic garbage collection                  |
| Automatic type conversion                     | No automatic type conversion                  |
| Has pointers                                  | NO pointers                                   |
| Different size variables / platform           | Similar sized variables / platform            |
| No boolean (uses integer)                     | True boolean variable                         |
| 8-bit char                                    | 16-bit char                                   |

# Java programming quirks

- Language evolution and deprecated features:
  - Java is constantly evolving, so some features are ‘deprecated’ at every release
  - But the same thing is true of most other languages
- Efficiency can be a concern in some object-oriented systems
  - Java can be less efficient than other languages
    - VM-based
    - Dynamic binding



# Why not C#?

- Proprietary language that runs optimally on Windows – possible to run on other platforms but requires work
- C# gets a bad reputation from its dirty little cousin VB
- Proprietary compilers are closed source for majority of the versions but available for free

# Why not stick with C++?

- Exposure to more languages to prepare you for FYP/real world
- You might not understand C++ that well so its better to start with a new language that tries to fix some of C++'s problems
- Opportunity to be certified as JCP or MCP – C++ has no certifications

# Revision!

- What you need to do before the next class:
  - Quick revision on C++ for the basic concepts
  - Java programming will be covered fairly quickly so your fundamentals should be strong enough
  - Basics will NOT be covered again (e.g. no more explaining what if-else is and what it does)

For next week:

- Download and install BlueJ <https://bluej.org/> for the tutorial
- If you have not programmed in Java with BlueJ before, do Tutorial 0